

MATLAB cheat sheet

ECON*6020 — Assignment 1: Canadian business cycle data, 1976Q1–2026Q2

Data file `Assignment1_data.xlsx`, sheet `Combined`: 202 quarters $\times$ 66 columns.

Companion to `assignment1.m`. This sheet lists the *syntax* you need — not the answers.

1 Load the file and look at it

```
clear; clc; close all

T = readtable("Assignment1_data.xlsx", Sheet = "Combined", ...
    VariableNamingRule = "preserve");

disp(head(T, 8))                % first 8 rows of the sheet
fprintf("%d quarters x %d columns\n", height(T), width(T));
var = string(T.Properties.VariableNames); % all 66 names as a string vector
q   = string(T.quarter);           % "1976Q1" ... "2026Q2" (column 1)
Tq  = height(T);                  % 202
```

`readtable` reads the sheet into a *table*; a column is then `T.name`. Things worth knowing: `height/width` give rows/columns (`size` works too), `head(T,n)` shows the first n rows, `T{:,k}` gives column k as a plain numeric vector, and `string(...)` converts to text so `==` comparisons work.

2 How the column names are built

Every GDP column name is three segments joined by `_`:

`basis _ seasonally_adjusted_at_annual_rates _ what`

where `basis` is `current_prices` (nominal) or `chained_2017_dollars` (real), and `what` is one of the 31 components:

<code>final_consumption_expenditure</code>	<code>exports_of_goods_and_services</code>
<code>household_final_consumption_expenditure</code>	<code>exports_of_goods</code>
<code>goods</code>	<code>exports_of_services</code>
<code>durable_goods</code>	<code>less_imports_of_goods_and_services</code>
<code>semi_durable_goods</code>	<code>imports_of_goods</code>
<code>non_durable_goods</code>	<code>imports_of_services</code>
<code>services</code>	<code>statistical_discrepancy</code>
<code>non_profit_institutions_serving_households_final_consumption_expenditure</code>	<code>gross_domestic_product_at_market_prices</code>
<code>general_governments_final_consumption_expenditure</code>	<code>final_domestic_demand</code>
<code>gross_fixed_capital_formation</code>	<code>investment_in_inventories</code>
<code>business_gross_fixed_capital_formation</code>	<code>of_which_business_investment_in_inventories</code>
<code>residential_structures</code>	<code>non_farm</code>
<code>non_residential_structures_machinery_and_equipment</code>	<code>farm</code>
<code>non_residential_structures</code>	
<code>machinery_and_equipment</code>	
<code>intellectual_property_products</code>	
<code>non_profit_institutions_serving_households_gross_fixed_capital_formation</code>	
<code>general_governments_gross_fixed_capital_formation</code>	

The three extra columns are `hours_worked_v4391505_quarterly_sum_hours`, `population_v1_persons` and `exchange_rate_v37426_to_2016_v111666275_from_2017_quarterly_average_cad_per_usd`.

Build a name instead of typing it

```

nm = @(basis, what) join([basis, "seasonally_adjusted_at_annual_rates", what], "_");
R  = "chained_2017_dollars";      % real, 2017 dollars
N  = "current_prices";           % nominal

y   = T.chained_2017_dollars_seasonally_adjusted_at_annual_rates_gross_domestic_product_at_market_prices;
ynom = T.current_prices_seasonally_adjusted_at_annual_rates_gross_domestic_product_at_market_prices;

% dot access works...
y   = T.(nm(R, "gross_domestic_product_at_market_prices"));
ynom = T.(nm(N, "gross_domestic_product_at_market_prices"));
% ...or computed

```

Every name is a legal MATLAB identifier

isvarname returns 1 for all 66 names (checked), so all three ways of reaching a column work:

```

pop = T.population_v1_persons;      % plain dot access
pop = T("population_v1_persons");  % same column, brackets
pop = T.(nm("chained_2017_dollars", "services")); % computed from the pieces

```

The GDP names run up to 129 characters, so the computed form is usually less typing; when you do type one out, copy it from T.Properties.VariableNames or from the header row of the Excel file.

3 The 14 variables of the table

```

% --- 10 series straight from the file -----
y   = T.(nm(R, "gross_domestic_product_at_market_prices")); % output, real
c   = T.(nm(R, "semi_durable_goods")) + ...
      T.(nm(R, "non_durable_goods")) + T.(nm(R, "services")); % consumption
i   = T.(nm(R, "non_residential_structures_machinery_and_equipment"));
gc  = T.(nm(R, "general_governments_final_consumption_expenditure"));
gi  = T.(nm(R, "general_governments_gross_fixed_capital_formation"));
ex  = T.(nm(R, "exports_of_goods_and_services"));
im  = T.(nm(R, "less_imports_of_goods_and_services")); % positive
n   = T.hours_worked_v4391505_quarterly_sum_hours; % hours
pop = T.population_v1_persons; % persons
fx  = 100 * T.exchange_rate_v37426_to_2016_v111666275_from_2017_quarterly_average_cad_per_usd;
      % CAD/USD -> cents

% --- 4 variables you build -----
nx  = 1 + (ex - im) ./ y; % net exports, 1 + NX/Y
w   = y ./ n; % labour productivity, Y/N
pgdp = 100 * ynom ./ y; % price = GDP deflator

delta = 0.025; % quarterly depreciation
gI    = mean(diff(i) ./ i(1:end-1)); % avg growth of investment
k     = zeros(Tq, 1);
k(1)  = i(1) / (gI + delta); % KO from (g+delta)K = I
for t = 1:Tq-1
    k(t+1) = i(t) + (1 - delta) * k(t); % law of motion
end
mpk   = y ./ k; % capital productivity, Y/K

% --- one matrix, table order, then per capita -----
X = [y c i gc gi ex im nx n w k mpk fx pgdp]; % 202 x 14
QC = [1 2 3 4 5 6 7 9 11]; % y c i gc gi ex im n k
X(:, QC) = X(:, QC) ./ pop; % ratios (nx w mpk) and indexes (fx pgdp) are
      % already per-capita, so leave them alone

```

Order of X (this is the table's order): 1 output, 2 consumption, 3 investment, 4 gov consumption, 5 gov investment, 6 exports, 7 imports, 8 net exports, 9 hours, 10 labour productivity, 11 capital, 12 capital

productivity, 13 nominal exchange rate, 14 price.

4 Operators: matrix algebra vs element-by-element

	matrix algebra	element-by-element
multiply	$A*B$	$A.*B$
divide	A/B (solve)	$A./B$
power	A^2	$A.^2$

Rule of thumb: two series of the same length are *always* `./` and `.*`. The plain versions are for matrix products, and using them by mistake gives either an error (*Incorrect dimensions*) or a silent scalar matrix product.

5 HP filter (the method in the course's `hpfilter.m`)

```
raw    = log(X);                                % log FIRST
lambda = 1600;
d = (1+6*lambda)*ones(Tq,1); x = diag(d);
d = -4*lambda*ones(Tq-1,1); x = x + diag(d,1) + diag(d,-1);
d = lambda*ones(Tq-2,1); x = x + diag(d,2) + diag(d,-2);
x(1,1) = 1+lambda; x(1,2) = -2*lambda; x(2,1) = -2*lambda;
x(2,2) = 1+5*lambda; x(Tq,Tq) = 1+lambda; x(Tq-1,Tq) = -2*lambda;
x(Tq,Tq-1) = -2*lambda; x(Tq-1,Tq-1) = 1+5*lambda;
hpmat = inv(x);                                % HP matrix, built once
trend = hpmat * raw;                           % trend, in logs
dev    = raw - trend;                           % cyclical component
```

The course file reads `data2026q2.csv` and derives the number of columns itself, so to reuse it exactly: `writematrix(X, "data2026q2.csv")` then run `hpfilter`. It hard-codes $T = 202$, which is our sample length, so the sizes work out.

6 Statistics

```
vcv = (dev' * dev) / Tq;                        % variance-covariance matrix (14x14)
v    = sum(vcv .* eye(14));                     % variances on the diagonal
sd   = sqrt(v) * 100;                           % std dev in percent
corr = vcv ./ sqrt(v' * v);                     % correlation matrix

corrWithY = corr(:, 1);                         % correlation with output (column 1)

% persistence of output: autocorrelation
y1 = dev(2:Tq, 1);                             % periods 2:T
y0 = dev(1:Tq-1, 1);                           % periods 1:T-1
ac  = corrcoeff(y1, y0);
rho = ac(1, 2);
```

`dev'*dev` multiplies the 202×14 matrix by its transpose, giving 14×14 . `sum(vcv .* eye(14))` keeps only the diagonal (the variances); `sqrt(v'*v)` is the outer product of the standard deviations, which is exactly the denominator of every correlation.

7 Plotting

```
t = (1:Tq)';
plot(t, dev(:,1), "LineWidth", 1.2); hold on
plot(t, dev(:,2), "LineWidth", 1.2); hold off
```

```
xticks(t(1:16:end)) % one tick every 4 years
xticklabels(q(1:16:end)) % ... labelled with the quarter strings
xlabel("quarter"); ylabel("log deviation from trend")
legend(["Output", "Consumption"], Location = "best"); grid on

figure
yyaxis left; plot(t, trend(:,1)); ylabel("trend")
yyaxis right; plot(t, dev(:,1)); ylabel("cycle")

exportgraphics(gcf, "fig.png", Resolution = 200) % or saveas(gcf, "fig.png")
```

`xticks` needs *numbers*, so pass positions (`t(1:16:end)`) and give the labels separately — calling `xticks` on the quarter strings errors.

8 Function quick reference

task	syntax	note
make a vector	<code>zeros(Tq,1)</code> , <code>ones(Tq,1)</code> , <code>(1:Tq)'</code>	<code>'</code> transposes
size	<code>size(X)</code> , <code>size(X,1)</code> , <code>height(T)</code>	<code>numel</code> for total count
mean / sum	<code>mean(x)</code> , <code>sum(x)</code> , <code>mean(M)</code>	on a matrix: column-wise
std	<code>std(x,1)</code>	1 = divide by N (population)
growth	<code>diff(x)./x(1:end-1)</code>	<code>diff</code> is one shorter
cumulate	<code>cumsum(x)</code>	
logs	<code>log(x)</code> , <code>exp(x)</code>	natural log
correlation	<code>corrcoef(a,b)</code> , <code>corr(M)</code>	2×2 / matrix
diagonal, identity	<code>diag(v)</code> , <code>eye(14)</code>	
inverse, solve	<code>inv(A)</code> , <code>A\b</code>	<code>A\b</code> is faster
indexing	<code>x(1)</code> , <code>X(:,k)</code> , <code>X(1:5,2)</code> , <code>x(end)</code>	starts at 1, not 0
text	<code>string(x)</code> , <code>strtrim(s)</code> , <code>join(v,"_")</code>	<code>==</code> works on strings
search in names	<code>contains(var,"goods")</code> , <code>startsWith</code> , <code>find(...)</code>	useful to find a column
print	<code>fprintf("%d rows\n", Tq)</code>	<code>%g</code> , <code>%.2f</code>
write files	<code>writematrix(X,"x.csv")</code> , <code>writetable</code>	
sanity check	<code>assert(numel(y)==Tq)</code> , <code>isnan(sum(y))</code>	catch size errors early
clear the deck	<code>clear</code> , <code>clc</code> , <code>close all</code>	start of every script

9 Errors I hit, and what fixes them

message / symptom	fix
<i>Invalid text character</i>	the name you typed contains something MATLAB cannot use (a space, a dash, a \$). Use <code>T.("...")</code> or fix the header
<i>VariableNames must be a character vector</i>	use <code>name=value</code> pairs: <code>table(a, VariableNames=["x"])</code> , not <code>table(a,"VariableNames",["x"])</code>
<i>Incorrect dimensions for matrix multiplication</i>	you meant <code>.*</code> or <code>./</code> instead of <code>*</code> / <code>/</code>
<i>xticks error on a categorical axis</i>	pass numeric positions, then <code>xticklabels(q(...))</code>
<i>height fails on a struct</i>	use <code>numel()</code> or keep the table instead of extracting fields
<i>results look absurd (huge trend swings)</i>	you skipped <code>log()</code> before filtering
<i>NaN in the first row of a growth series</i>	expected: <code>diff</code> shortens the series; index with <code>(1:end-1)</code>
<i>numbers off by one quarter</i>	MATLAB indexes from 1, an "0-based" habit from other languages
<i>script name starts with _ or a digit</i>	MATLAB scripts need a valid identifier as filename

10 The whole pipeline, in order

```
load and inspect -> build the 14 series -> per capita -> log
-> HP filter (trend + cycle) -> variance-covariance -> std dev
-> correlations with output -> autocorrelation of output
-> plot trend & cycle -> write the table / figure out
```

Keep the steps in separate cells (or %% sections) so you can rerun from the middle without recomputing everything.